

Модуль 8. Архитектура очередей задач на базе Celery и Redis

8.1. Зачем нужен Celery: от ручных команд к декораторам

В Модуле 7 мы научились работать с Redis Streams напрямую: писали `XADD` вручную, читали через `XREAD`, управляли группами и отправляли `XACK`. Это было полезно, потому что мы увидели, как устроен брокер «изнутри». Но в реальной разработке писать такой код для каждой задачи — всё равно что строить дом из отдельных кирпичей, привозя каждый на тачке, вместо того чтобы заказать бетономешалку и подъёмный кран.

Celery — это как раз «стройтехника» для очередей задач. Это высокоуровневая Python-библиотека, которая берёт на себя всю рутину:

- Сериализацию аргументов функции в сообщение.
- Отправку сообщения в брокер.
- Получение сообщения воркером.
- Вызов вашей функции с правильными аргументами.
- Подтверждение выполнения (ACK).
- Хранение результата.
- Повторные попытки при ошибках (retry).
- Мониторинг и логирование.

Аналогия: ресторан с автоматизацией

В Модуле 6 мы моделировали ресторан вручную: официант писал заказ на бумажке, нёс на кухню, клал в стопку. Повар брал из стопки, готовил, кричал «Готово!».

Celery — это как **автоматизированный ресторан**:

- Вы говорите: «Приготовь борщ» — и система сама печатает чек, отправляет его на кухню, находит свободного повара, следит, чтобы блюдо не потерялось, и сообщает вам, когда можно забирать.
- Вам не нужно знать, какой у повара номер, в какой он смене, и где именно лежит стопка заказов. Вы просто говорите: «`.delay()`» — и задача улетает.

8.2. Архитектура Celery: три столпа

Любая система на Celery состоит из трёх обязательных частей и одной опциональной.

8.2.1. Message Broker (Посредник, брокер сообщений)

Роль: Принимает сообщения от Producer'a (вашего API) и хранит их до тех пор, пока Worker не заберёт.

В нашем курсе: Redis. Celery использует Redis как транспорт. Важно: по умолчанию Celery использует **Redis Lists** (`LPUSH / BRPOP`), а не Redis Streams. Это эфемерная очередь: сообщение удаляется из списка, когда Worker его забирает. Это нормально для задач — нам не нужна вечная история, нам нужно выполнить и забыть.

Аналогия: Брокер — это стопка заказов на кухне. Официант кладёт сверху, повар забирает снизу.

8.2.2. Worker (Воркер, потребитель)

Роль: Процесс (или несколько процессов), который постоянно слушает брокер. Как только появляется задача — Worker забирает её и выполняет вашу Python-функцию.

Аналогия: Повар на кухне. Он стоит у стопки заказов, берёт верхний, готовит, кричит «Готово!», берёт следующий.

Важно: Worker — это **отдельный процесс**. Он не работает внутри вашего FastAPI-приложения. Обычно его запускают в отдельном терминале или отдельном контейнере.

8.2.3. Producer (Производитель)

Роль: Тот, кто создаёт задачи. В нашем случае — это endpoint FastAPI. Когда пользователь нажимает «Обучить модель», API не выполняет обучение самостоятельно, а говорит Celery: «Поставь эту задачу в очередь».

Аналогия: Официант. Он не готовит еду, он только принимает заказ и несёт на кухню.

8.2.4. Result Backend (Хранилище результатов) — опционально, но критично для ML

Роль: Хранит **результат** выполнения задачи. Когда Worker закончил работу, он записывает результат (например, ассурасу модели, путь к файлу, или ошибку) в Result Backend. Затем Producer может спросить: «А что там с задачей №42?»

В нашем курсе: Тоже Redis (или PostgreSQL для надёжности). Если использовать Redis и для Broker, и для Backend — это удобно, но в продакшене иногда разделяют.

Аналогия: Полка с готовыми заказами. Повар положил блюдо на полку и повесил номерок. Клиент подходит с номерком и забирает результат. Если блюдо ещё готовится — полка пуста.

8.3. Установка и создание проекта

Шаг 1. Подготовка окружения

Создайте рабочую папку:

```
In [1]: mkdir ~/docker-module8
        cd ~/docker-module8

        python -m venv venv
        source venv/bin/activate # Linux/macOS
        # или venv\Scripts\activate # Windows
```

Шаг 2. Установка зависимостей

```
In [2]: pip install celery[redis] redis fastapi uvicorn
```

Разбор:

- `celery[redis]` — Celery с поддержкой Redis (устанавливает `redis-py` и дополнительные зависимости).
- `redis` — клиент для Python (хотя `[redis]` уже тянет его, явное указание не мешает).
- `fastapi`, `uvicorn` — для API.

Шаг 3. Запуск Redis

Если Redis не запущен:

```
In [3]: docker run -d --name redis_celery -p 6379:6379 redis:7-alpine
```

8.4. Создание Celery-приложения: конфигурация

Создайте файл `celery_app.py` — это «сердце» вашей очереди задач.

```
In [4]: # celery_app.py
        from celery import Celery

        # Создаём экземпляр Celery
        # Первый аргумент — имя проекта (используется в логах)
        # broker — URL к Redis, который будет брокером сообщений
        # backend — URL к Redis, который будет хранилищем результатов
        app = Celery(
            "ml_tasks",
```

```

broker="redis://localhost:6379/0",
backend="redis://localhost:6379/0",
include=["tasks"] # список модулей, где искать задачи
)

# Опциональные настройки
app.conf.update(
    # Сериализация задач и результатов в JSON
    task_serializer="json",
    accept_content=["json"],
    result_serializer="json",

    # Часовой пояс (важно для отложенных задач)
    timezone="UTC",
    enable_utc=True,

    # Результаты храним 1 час (3600 секунд), потом удаляем
    # Это важно: Redis — память, не надо захламлять
    result_expires=3600,

    # Если задача упала, повторить максимум 3 раза
    task_max_retries=3,

    # Первый retry через 60 секунд
    default_retry_delay=60,
)

```

Разбор строк:

```
broker="redis://localhost:6379/0"
```

- `redis://` — протокол.
- `localhost:6379` — хост и порт.
- `/0` — номер логической базы данных Redis (Redis поддерживает 16 БД по умолчанию, от 0 до 15).

```
backend="redis://localhost:6379/0"
```

Тот же Redis, но можно было бы указать другой инстанс или PostgreSQL.

```
include=["tasks"]
```

Celery будет искать декорированные функции (`@app.task`) в файле `tasks.py` . Это позволяет разделить конфигурацию и бизнес-логику.

8.5. Написание задачи: декоратор `@app.task`

Создайте файл `tasks.py` в той же папке.

```
In [5]: # tasks.py
import time
import random
from celery_app import app

@app.task(bind=True)
def train_model(self, dataset_path: str, model_type: str = "linear"):
    """
    Задача обучения ML-модели.
    Выполняется в фоне, может длиться минутами.

    bind=True даёт нам доступ к self – объекту задачи Celery.
    Через self мы можем обновлять статус, логировать прогресс.
    """
    print(f"[WORKER] Начинаю обучение модели {model_type} на данных {dataset_path}")

    # Сообщаем прогресс: задача в процессе
    self.update_state(state="PROGRESS", meta={"progress": 10, "status": "Загрузка данных"})
    time.sleep(2)

    # Имитация предобработки
    self.update_state(state="PROGRESS", meta={"progress": 40, "status": "Предобработка признаков"})
    time.sleep(3)

    # Имитация обучения
    self.update_state(state="PROGRESS", meta={"progress": 80, "status": "Обучение модели"})
    time.sleep(3)

    # Имитация валидации
    accuracy = random.uniform(0.75, 0.95)
    self.update_state(state="PROGRESS", meta={"progress": 95, "status": "Валидация"})
    time.sleep(1)

    print(f"[WORKER] Обучение завершено. Accuracy: {accuracy:.4f}")

    # Возвращаем результат – он попадёт в Result Backend
    return {
```

```

        "status": "completed",
        "model_type": model_type,
        "dataset": dataset_path,
        "accuracy": round(accuracy, 4),
        "model_path": f"/models/{model_type}_{int(time.time())}.pkl"
    }

@app.task(bind=True)
def preprocess_data(self, raw_data_path: str):
    """
    Задача предобработки данных.
    """
    print(f"[WORKER] Предобработка файла {raw_data_path}")
    time.sleep(2)

    # Имитация ошибки с вероятностью 20% (для демо retry)
    if random.random() < 0.2:
        raise ValueError("Ошибка чтения файла: повреждённые данные")

    return {
        "status": "completed",
        "input": raw_data_path,
        "output": raw_data_path.replace("raw", "processed"),
        "rows_processed": random.randint(1000, 50000)
    }

```

Разбор декоратора `@app.task(bind=True)` :

- `@app.task` — регистрирует функцию как Celery-задачу. Celery узнаёт о ней и может отправлять в брокер.
- `bind=True` — «привяжи задачу к экземпляру». Внутри функции появляется параметр `self` — это объект `Task`, через который можно:
 - `self.update_state(...)` — обновить статус (для прогресса).
 - `self.retry(...)` — попросить повторить задачу при ошибке.
 - `self.request.id` — получить уникальный ID задачи.

Почему `time.sleep()` ?

В реальности здесь был бы `model.fit(X, y)`, чтение гигабайтов данных, запросы к внешним API. Мы используем `sleep`, чтобы имитировать долгую работу без необходимости тащить тяжёлые ML-библиотеки.

8.6. Отправка задач: `.delay()` vs `.apply_async()`

Теперь научимся ставить задачи в очередь из Python-кода.

8.6.1. `.delay()` — простой способ

```
In [6]: from tasks import train_model

# Отправить задачу в очередь
result = train_model.delay("s3://bucket/dataset_v1.csv", "random_forest")
```

Что происходит:

1. Celery сериализует аргументы (`"s3://bucket/dataset_v1.csv"` , `"random_forest"`) в JSON.
2. Создаёт сообщение с уникальным ID (например, `a1b2-c3d4-e5f6`).
3. Кладёт сообщение в Redis (в список, соответствующий очереди `celery`).
4. Немедленно возвращает объект `AsyncResult` — «обещание» результата.
5. Ваш код продолжает работать, не дожидаясь выполнения.

Что возвращает `.delay()` :

Объект `AsyncResult` . У него есть атрибут `.id` — уникальный идентификатор задачи. Этот ID вы можете показать пользователю.

8.6.2. `.apply_async()` — продвинутый способ

```
In [7]: result = train_model.apply_async(
    args=["s3://bucket/dataset_v1.csv"],
    kwargs={"model_type": "neural"},
    countdown=60,           # выполнить через 60 секунд
    expires=3600,           # если не выполнена за час — отменить
    retry=True,             # повторять при ошибке
    retry_policy={
        "max_retries": 5,
        "interval_start": 30,
        "interval_step": 30,
        "interval_max": 300,
    }
)
```

Когда использовать:

- `countdown` / `eta` — отложенные задачи («отправь письмо завтра в 9:00»).
- `expires` — задачи, которые актуальны только ограниченное время.
- `retry_policy` — тонкая настройка повторных попыток.

8.6.3. Практика: отправляем задачи из скрипта

Создайте `send_tasks.py` :

```
In [8]: # send_tasks.py
from tasks import train_model, preprocess_data

def main():
    print("=" * 50)
    print("ОТПРАВКА ЗАДАЧ В CELERY")
    print("=" * 50)

    # Задача 1: предобработка
    job1 = preprocess_data.delay("raw_data/customers_2024.csv")
    print(f"[API] Отправлена задача предобработки, ID: {job1.id}")

    # Задача 2: обучение
    job2 = train_model.delay("processed_data/features_v2.csv", "xgboost")
    print(f"[API] Отправлена задача обучения, ID: {job2.id}")

    # Задача 3: обучение с отложенным стартом (через 10 сек)
    job3 = train_model.apply_async(
        args=["processed_data/features_v3.csv"],
        kwargs={"model_type": "neural"},
        countdown=10
    )
    print(f"[API] Отложенная задача обучения, ID: {job3.id}")

    print()
    print("Задачи отправлены. API свободен!")
    print("Проверьте статус через Result Backend по этим ID.")

if __name__ == "__main__":
    main()
```

Запустите (пока Worker НЕ запущен — это нормально):


```
In [9]: python send_tasks.py
```

Вы увидите три ID задач. Они лежат в Redis и ждут своего Worker'a.

8.7. Запуск воркера и наблюдение

8.7.1. Команда запуска

```
In [10]: celery -A celery_app worker --loglevel=info
```

Разбор:

- `celery` — команда.
- `-A celery_app` — указать модуль с приложением (`celery_app.py`).
- `worker` — запустить воркер.
- `--loglevel=info` — уровень логирования.

Что вы увидите в терминале:

```
In [11]: ----- celery@your-laptop v5.4.0
-- ***** -----
-- ***** ----- Linux-5.15.0-x86_64 2026-08-20 12:00:00
- *** --- * ---
- ** ----- [config]
- ** ----- .> app:          ml_tasks:0x7f8b2c3d4e5f
- ** ----- .> transport:   redis://localhost:6379/0
- ** ----- .> results:    redis://localhost:6379/0
- *** --- * --- .> concurrency: 8 (prefork)
-- ***** -----
----- [queues]
          .> celery          exchange=celery(direct) key=celery
```

Важные строки:

- `transport: redis://localhost:6379/0` — брокер.
- `results: redis://localhost:6379/0` — бэкенд результатов.
- `concurrency: 8` — воркер запустит 8 процессов-потребителей (по умолчанию = число ядер CPU).

8.7.2. Что происходит при запуске

1. Worker подключается к Redis.
2. Слушает очередь `celery` (список в Redis).
3. Если там есть задачи — немедленно забирает и выполняет.
4. Если задач нет — блокируется (`BRPOP`), ждёт новых.

Запустите `send_tasks.py` ещё раз (в другом терминале) и наблюдайте, как Worker «съедает» задачи.

8.8. Механизм подтверждения задач: ACK и NACK

Это один из самых важных разделов. Здесь мы разберём, как Celery гарантирует, что задача не потеряется.

8.8.1. Что такое ACK (Acknowledgment)

ACK — это сигнал от Worker'а брокеру: «Я успешно выполнил задачу, удали её из очереди».

Аналогия: Повар приготовил блюдо, отнёс клиенту, клиент подписал чек. Повар возвращается на кухню и сжигает бумажку с заказом: «Готово, больше не нужно».

Когда происходит ACK в Celery:

По умолчанию — **после успешного выполнения** задачи. То есть Worker:

1. Забирает сообщение из Redis.
2. Выполняет функцию `train_model(...)`.
3. Если функция завершилась без исключения — отправляет ACK в Redis.
4. Redis удаляет сообщение из очереди.

8.8.2. Что такое NACK (Negative Acknowledgment)

NACK — это сигнал: «Я НЕ смог выполнить задачу, верни её в очередь (или отправь в мёртвую очередь)».

Аналогия: Повар начал готовить блюдо, но обнаружил, что мясо протухло. Он кричит менеджеру: «Брак! Верни заказ в очередь, пусть другой повар попробует, или пусть официант предупредит клиента».

Когда происходит NACK в Celery:

- Если внутри задачи произошло **исключение**, и исчерпаны все попытки `retry`.
- Или если Worker упал (был убит процесс) **во время выполнения** задачи.

8.8.3. Критически важный нюанс: visibility timeout

Представьте: Worker забрал задачу из Redis и начал выполнять. На это ушло 10 минут. Но на 9-й минуте Worker упал (Out of Memory, перезагрузка сервера).

Что должно произойти?

Задача должна вернуться в очередь, чтобы другой Worker мог её выполнить.

Как это работает:

Celery (через Redis) использует механизм, похожий на **visibility timeout** (таймаут видимости):

- Когда Worker забирает задачу, сообщение временно исчезает из очереди.
- Worker должен либо выполнить задачу и отправить ACK, либо...
- Если ACK не пришёл в течение определённого времени — сообщение **возвращается** в очередь.

Важно: По умолчанию в Celery с Redis ACK отправляется **ДО** выполнения (`task_acks_late = False`). Это означает, что если Worker упал во время выполнения — задача **потеряна!** Для надёжных ML-задач это плохо.

Исправление: Включить «поздний ACK».

```
In [12]: # celery_app.py
app.conf.update(
    task_acks_late=True, # ACK только после УСПЕШНОГО выполнения
    task_reject_on_worker_lost=True, # Если воркер умер — вернуть задачу
)
```

Сравнение:

Режим	ACK когда	Если Worker упал во время задачи
<code>acks_early</code> (по умолчанию)	Сразу после получения	Задача потеряна
<code>acks_late=True</code>	После успешного выполнения	Задача возвращается в очередь

Рекомендация для ML: Всегда используйте `task_acks_late=True` . Обучение модели может занимать минуты, и падение воркера — не редкость.

8.9. Result Backend: хранение и получение результатов

8.9.1. Зачем нужен Result Backend

Когда API отправляет задачу через `.delay()`, он получает только **ID**. Через некоторое время пользователь (или сам API) хочет узнать: «А что там с моей задачей?»

Result Backend отвечает на вопросы:

- Задача ещё в очереди? (`PENDING`)
- Выполняется? (`PROGRESS` — если вы обновляете вручную)
- Успешно завершена? (`SUCCESS`)
- Упала с ошибкой? (`FAILURE`)

8.9.2. Как проверить статус задачи

Создайте `check_status.py` :

```
In [13]: # check_status.py
from celery.result import AsyncResult
from celery_app import app

def check_task(task_id: str):
    """Проверяем статус задачи по её ID."""
    result = AsyncResult(task_id, app=app)

    print(f"ID задачи: {task_id}")
    print(f"Статус: {result.status}")
    print(f"Готова: {result.ready()}")
    print(f"Успешна: {result.successful()}")
    print(f"Провалена: {result.failed()}")

    if result.successful():
        print(f"Результат: {result.result}")
    elif result.failed():
        print(f"Ошибка: {result.traceback}")
    else:
        print("Результат ещё не готов.")

    print("-" * 40)
```

```

if __name__ == "__main__":
    # Подставьте реальные ID из вывода send_tasks.py
    task_ids = [
        "a1b2c3d4-e5f6-7890-abcd-ef1234567890", # замените на свой
        "b2c3d4e5-f6a7-8901-bcde-f12345678901", # замените на свой
    ]

    for tid in task_ids:
        check_task(tid)

```

Запускайте несколько раз, пока Worker выполняет задачи, и наблюдайте, как меняется статус:

- PENDING -> PROGRESS (если вы обновляете) -> SUCCESS .

8.9.3. Где хранятся результаты

В Redis результаты хранятся как ключи вида:

```
In [14]: celery-task-meta-a1b2c3d4-e5f6-7890-abcd-ef1234567890
```

Значение — сериализованный JSON со статусом, результатом и traceback.

8.10. Практика: полноценная интеграция с FastAPI

Теперь соберём всё вместе. Создадим API, которое:

1. Принимает POST-запрос на обучение модели.
2. Ставит задачу в Celery.
3. Сразу возвращает клиенту ID задачи.
4. Предоставляет endpoint для проверки статуса по ID.

Шаг 1. API-файл (api.py)

```
In [15]: # api.py
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from celery.result import AsyncResult

from tasks import train_model, preprocess_data
from celery_app import app as celery_app

```

```
app = FastAPI(title="ML Task Queue API")

# --- Модели данных ---

class TrainRequest(BaseModel):
    dataset_path: str
    model_type: str = "linear"

class TaskResponse(BaseModel):
    task_id: str
    status: str
    message: str

class TaskStatusResponse(BaseModel):
    task_id: str
    status: str
    result: dict | None = None
    error: str | None = None

# --- Endpoints ---

@app.post("/train", response_model=TaskResponse)
def start_training(request: TrainRequest):
    """
    Запускает обучение модели в фоне.
    Возвращает ID задачи немедленно, не дожидаясь результата.
    """
    task = train_model.delay(request.dataset_path, request.model_type)

    return TaskResponse(
        task_id=task.id,
        status="queued",
        message=f"Задача на обучение {request.model_type} поставлена в очередь"
    )

@app.post("/preprocess", response_model=TaskResponse)
def start_preprocess(dataset_path: str):
    """Запускает предобработку данных."""
    task = preprocess_data.delay(dataset_path)
```

```

    return TaskResponse(
        task_id=task.id,
        status="queued",
        message="Задача предобработки поставлена в очередь"
    )

@app.get("/status/{task_id}", response_model=TaskStatusResponse)
def get_status(task_id: str):
    """Проверяет статус задачи по ID."""
    task_result = AsyncResult(task_id, app=celery_app)

    response = TaskStatusResponse(
        task_id=task_id,
        status=task_result.status,
        result=None,
        error=None
    )

    if task_result.ready():
        if task_result.successful():
            response.result = task_result.result
        else:
            response.error = str(task_result.result) # здесь будет Exception

    return response

@app.get("/")
def root():
    return {
        "service": "ML Task Queue",
        "endpoints": {
            "train": "POST /train",
            "preprocess": "POST /preprocess",
            "status": "GET /status/{task_id}"
        }
    }

```

Шаг 2. Запуск системы

Нужно запустить **три** вещи в **разных** терминалах:

Терминал 1 — Redis (если не запущен):

```
In [16]: docker run -d --name redis_celery -p 6379:6379 redis:7-alpine
```

Терминал 2 — Worker:

```
In [17]: cd ~/docker-module8
source venv/bin/activate
celery -A celery_app worker --loglevel=info
```

Терминал 3 — API:

```
In [18]: cd ~/docker-module8
source venv/bin/activate
uvicorn api:app --reload --port 8000
```

Шаг 3. Тестирование через curl или браузер

```
In [19]: # Запускаем обучение
curl -X POST "http://localhost:8000/train" \
  -H "Content-Type: application/json" \
  -d '{"dataset_path": "s3://ml-bucket/customers.csv", "model_type": "random_forest"}'

# Ответ:
# {"task_id": "a1b2-c3d4-...", "status": "queued", "message": "..."}

# Проверяем статус (подставьте реальный task_id)
curl "http://localhost:8000/status/a1b2-c3d4-..."

# Пока воркер работает — статус будет PROGRESS или PENDING
# Когда закончит — SUCCESS с результатом (accuracy и т.д.)
```

Наблюдайте:

- API отвечает мгновенно (50 мс), даже если обучение длится 10 секунд.
- Worker в терминале 2 показывает прогресс выполнения.
- Статус можно опрашивать сколько угодно раз по `task_id`.

8.11. Retry и обработка ошибок

В `tasks.py` мы уже добавили имитацию ошибки в `preprocess_data`. Давайте разберём, как Celery справляется с падениями.

8.11.1. Автоматический `retry`

Если в задаче возникает исключение, Celery может автоматически повторить попытку.

```
In [20]: @app.task(bind=True, max_retries=3, default_retry_delay=10)
def fragile_task(self, data):
    try:
        # какая-то опасная операция
        result = 1 / data # если data=0, будет ZeroDivisionError
        return result
    except ZeroDivisionError as exc:
        # Повторить через 10 секунд
        raise self.retry(exc=exc, countdown=10)
```

8.11.2. Настройки уровня приложения

В `celery_app.py` мы уже указали:

```
In [21]: task_max_retries=3,
default_retry_delay=60,
```

Это означает: если задача упала, и в коде нет явного `self.retry()` — Celery всё равно повторит её 3 раза с интервалом 60 секунд.

8.11.3. Что происходит при исчерпании попыток

Если все `retry` исчерпаны — задача получает статус `FAILURE`. Результат в Result Backend содержит traceback ошибки. API может показать пользователю: «Задача не выполнена. Обратитесь в поддержку».

8.12. Под капотом: как Celery говорит с Redis (упрощённо)

Для полноты картины полезно понимать, что происходит «под капотом», хотя Celery это скрывает.

8.12.1. Брокер: Redis Lists

Когда вы вызываете `.delay()`:

1. Celery сериализует аргументы в JSON: `'["s3://bucket/data.csv", "xgboost"]'` .
2. Создаёт сообщение с метаданными: ID задачи, имя функции, аргументы, ETA.
3. Выполняет `LPUSH celery <сообщение>` — кладёт в конец списка `celery` в Redis.

Когда Worker запущен:

1. Он выполняет `BRPOP celery 1` — блокирующее чтение с конца списка.
2. Как только сообщение появляется — Worker забирает его.
3. Десериализует JSON, импортирует функцию `train_model` , вызывает её.

8.12.2. Result Backend: Redis Strings

Когда Worker заканчивает:

1. Сериализует результат в JSON.
2. Выполняет `SET celery-task-meta-<task_id> <json_result> EX 3600` .
3. `EX 3600` — ключ живёт 1 час, потом автоматически удаляется (чтобы Redis не переполнился).

8.12.3. Почему это не Streams

По умолчанию Celery использует **Lists**, потому что ему нужна классическая эфемерная очередь: задача выполняется один раз и забывается. Нет нужды хранить историю всех задач — это было бы расточительно по памяти.

Redis Streams поддерживается Celery как экспериментальный транспорт, но для задач обучения моделей классические списки проще и эффективнее.

8.13. Итоги модуля: чек-лист

- ☐ Понимаю, зачем нужен Celery: высокоуровневая абстракция над брокером.
- ☐ Знаю три столпа архитектуры: **Broker** (Redis), **Worker** (воркер), **Result Backend** (Redis).
- ☐ Умею создавать Celery-приложение и конфигурировать его (`broker` , `backend` , `result_expires`).
- ☐ Знаю, что `include=["tasks"]` сообщает Celery, где искать задачи.
- ☐ Умею создавать задачу через декоратор `@app.task(bind=True)` .
- ☐ Знаю разницу между `.delay()` (просто) и `.apply_async()` (гибко).
- ☐ Умею запускать Worker командой `celery -A celery_app worker` .
- ☐ Понимаю механизм **ACK**: сигнал об успешном выполнении, по которому сообщение удаляется.

- ☐ Понимаю механизм **NACK/возврата**: если Worker упал, задача может вернуться в очередь.
- ☐ Знаю про `task_acks_late=True` и почему он критичен для долгих ML-задач.
- ☐ Умею проверять статус задачи через `AsyncResult(task_id)` и понимаю статусы: `PENDING`, `SUCCESS`, `FAILURE`, `PROGRESS`.
- ☐ Собрал полноценное FastAPI + Celery + Redis приложение, которое принимает задачи, возвращает ID и позволяет отслеживать прогресс.
- ☐ Понимаю, как Celery использует Redis Lists под капотом для эфемерных сообщений.
- ☐ Знаю про `retry` и умею настраивать повторные попытки.

В следующем модуле мы сделаем архитектурный скачок: разберём **отличия очередей задач** (эфемерные сообщения в Celery/Redis Lists) от **логов событий** (персистентные, неизменяемые сообщения в Kafka). Это подготовит нас к изучению Apache Kafka.